

OmniFlow

Enterprise-Grade DevOps Pipeline & Infrastructure

Continuous Assessment Project Documentation

Version 1.0 | May 2026

1. Executive Summary

OmniFlow is a demonstration of end-to-end DevOps maturity. The application itself is a simple 3-Tier Task Manager (Frontend, Backend API, Database). However, the infrastructure and deployment pipeline surrounding it are built to enterprise standards, featuring High Availability (HA), Zero-Downtime Deployments, Infrastructure as Code (IaC), advanced CI/CD gates, security scanning, and full observability.

CALMS Framework Alignment

This project fully implements the CALMS framework (Culture, Automation, Lean, Measurement, Sharing) and solves the 'Wall of Confusion' by ensuring that environment mismatches, manual configurations, and 'Big Bang' releases are entirely eliminated through automation.

2. Application Architecture

The application consists of three simple microservices, containerized and orchestrated. The intentional simplicity of the application code demonstrates the core DevOps principle: complexity lives in the infrastructure and pipeline, not in the application itself.

2.1 Services Overview

- **Frontend**
 - Technology: React.js or Vue.js
 - Deployed as a static web application
 - Served via NGINX Ingress Controller
- **Backend API**
 - Technology: Node.js/Express or Python/FastAPI

- Exposes a RESTful API with a /metrics endpoint for Prometheus
- Stateless — horizontally scalable
- **Database & Cache**
 - Primary store: PostgreSQL (managed, with daily automated backups)
 - Cache layer: Redis (for session management and response caching)

3. DevOps Architecture & Toolchain

The following toolchain covers every stage of the DevOps lifecycle, from source control through to production observability.

Phase	Tool	Purpose
Version Control	Git / GitHub	Trunk-based development & Source of Truth
CI/CD	GitHub Actions & Jenkins	GitHub Actions for CI (Build/Test/Scan), Jenkins for CD (Advanced GitOps deployment)
Containerization	Docker & Docker Compose	Multi-stage builds for optimized, secure production images
Orchestration	Kubernetes (K8s)	Deployment, scaling, and managing containerized microservices
Package Management	Helm	Templating and managing K8s manifests across environments
Infrastructure as Code	Terraform	Provisioning cloud resources (AWS/DigitalOcean)
Monitoring & Logging	Prometheus, Grafana, Loki	Time-series metrics, visualization, and centralized log aggregation
Security	Trivy, SonarQube, Vault	Container scanning, static code analysis, and secrets management

4. Infrastructure as Code (IaC) with Terraform

Following the principle of 'No Snowflake Servers,' all infrastructure is defined as code. This guarantees idempotency — running Terraform 100 times produces the exact same outcome with no side effects.

4.1 Terraform Modules

- networking — VPC, Subnets, Route Tables, Internet Gateways
- kubernetes — EKS (AWS) or DOKS (DigitalOcean) cluster provisioning
- database — Managed PostgreSQL instance

- storage — S3 buckets for Terraform state and frontend static hosting

4.2 Remote State Management

Terraform state is stored in a remote, encrypted S3 bucket with DynamoDB locking to prevent concurrent infrastructure changes — a critical requirement in team environments.

4.3 Environment Parity

Separate Terraform workspaces (dev, staging, prod) ensure that all environments are identical clones, definitively solving the 'It works on my machine' problem. Configuration differences between environments are managed entirely through Terraform variable files.

5. CI/CD Pipeline Design

The pipeline implements Continuous Deployment, meaning every change that passes all automated gates goes live with zero human intervention. The pipeline is split across two tools to provide clear separation of concerns.

5.1 Phase 1 — Continuous Integration (GitHub Actions)

Triggered on every Pull Request or push to the main branch.

1. Linting & Static Analysis — ESLint / Pylint enforce code style and detect bugs early.
2. SAST Security Scan — SonarQube scans source code for vulnerabilities (Shift-Left Security: find issues before they reach staging).
3. Unit & Integration Tests — Automated tests run via Jest (frontend) and Pytest (backend).
4. Container Build — Docker images are built using multi-stage builds, reducing image size and attack surface.
5. Container Scan — Trivy scans images for OS and package CVEs. The pipeline fails automatically if critical vulnerabilities are found.
6. Push to Registry — Verified, tagged images are pushed to Docker Hub or Amazon ECR.

5.2 Phase 2 — Continuous Deployment (Jenkins + Helm)

Triggered after the CI phase passes and changes are merged to main.

7. Helm Chart Update — Jenkins bumps the image tag in the Helm values.yaml file.
8. GitOps Commit — Jenkins commits the updated chart to the Git configuration repository. Git is the Single Source of Truth for cluster state.
9. Staging Deployment — Jenkins triggers a Helm upgrade on the Staging Kubernetes cluster.

10. Automated Smoke Tests — Cypress end-to-end tests validate the staging deployment before promotion.
11. Production Deployment (Zero-Downtime) — Helm applies the release via Rolling Updates. Kubernetes Readiness and Liveness probes ensure new pods are healthy before old pods are terminated.
12. Automatic Rollback — If deployment fails at any gate, Jenkins automatically executes helm rollback.

6. Kubernetes & Helm Architecture

All Kubernetes manifests are managed through Helm to enable parameterised, repeatable deployments across environments without duplicating YAML.

6.1 Helm Chart Structure

One umbrella Helm chart containing sub-charts for frontend, backend, and redis. This ensures all services are versioned, deployed, and rolled back atomically.

6.2 Kubernetes Manifest Inventory

- Deployments — Pod replicas, CPU/RAM resource limits, and security contexts.
- Services — ClusterIP for internal service communication; LoadBalancer for external ingress traffic.
- Ingress — NGINX Ingress Controller routing: /api routes to Backend, / routes to Frontend.
- Horizontal Pod Autoscaler (HPA) — Automatically scales backend pods when CPU exceeds 70%, ensuring the application handles traffic spikes without manual intervention.
- Secrets — Managed via HashiCorp Vault or AWS Secrets Manager; never stored as hardcoded base64 strings in Git.

7. Observability & Monitoring

Core Principle

You cannot fix what you cannot see. The full observability stack is deployed via Helm alongside the application, ensuring monitoring is never an afterthought.

7.1 Metrics — Prometheus

Prometheus scrapes time-series metrics from Kubernetes nodes, pods, and the application's /metrics endpoint. Metrics are retained for 15 days and queried using PromQL.

7.2 Dashboards — Grafana

- Pre-configured Kubernetes cluster health dashboard (node CPU, memory, disk, network).
- Application-specific dashboard tracking HTTP requests per minute, error rates (5xx), and p99 latency.

7.3 Logging — Loki

Loki aggregates logs from all containers. Engineers can query logs directly in Grafana without SSH access to any server, maintaining the immutable infrastructure principle.

7.4 Alerting — Alertmanager

Configured to send Slack and Discord notifications for the following conditions:

- Pod enters CrashLoopBackOff state.
- CPU or Memory exceeds 90% for more than 5 consecutive minutes.
- TLS/SSL certificate expiration is fewer than 14 days away.

8. DevOps Best Practices Implemented

The following best practices are explicitly implemented and can be cited in the CA report to justify design decisions.

8.1 Infrastructure as Code (Idempotency)

Running terraform apply 100 times produces the same infrastructure state. No manual console clicks; no snowflake servers; no configuration drift.

8.2 Shift-Left Security

Security scanning (Trivy, SonarQube) occurs during CI — before code ever reaches a deployment environment. Vulnerabilities are caught at the cheapest possible point in the lifecycle.

8.3 Zero-Downtime Releases

Kubernetes Rolling Updates combined with Readiness and Liveness probes eliminate the 'Friday Night Disaster.' Users experience no downtime during deployments.

8.4 Immutable Infrastructure

Servers are never patched or configured in place. Every change produces a new Docker image, which replaces old pods. SSH access to production is not required and is actively disabled.

8.5 Trunk-Based Development

Engineers commit frequently to short-lived feature branches that are merged to main rapidly. This eliminates 'Merge Hell' and keeps the integration overhead low.

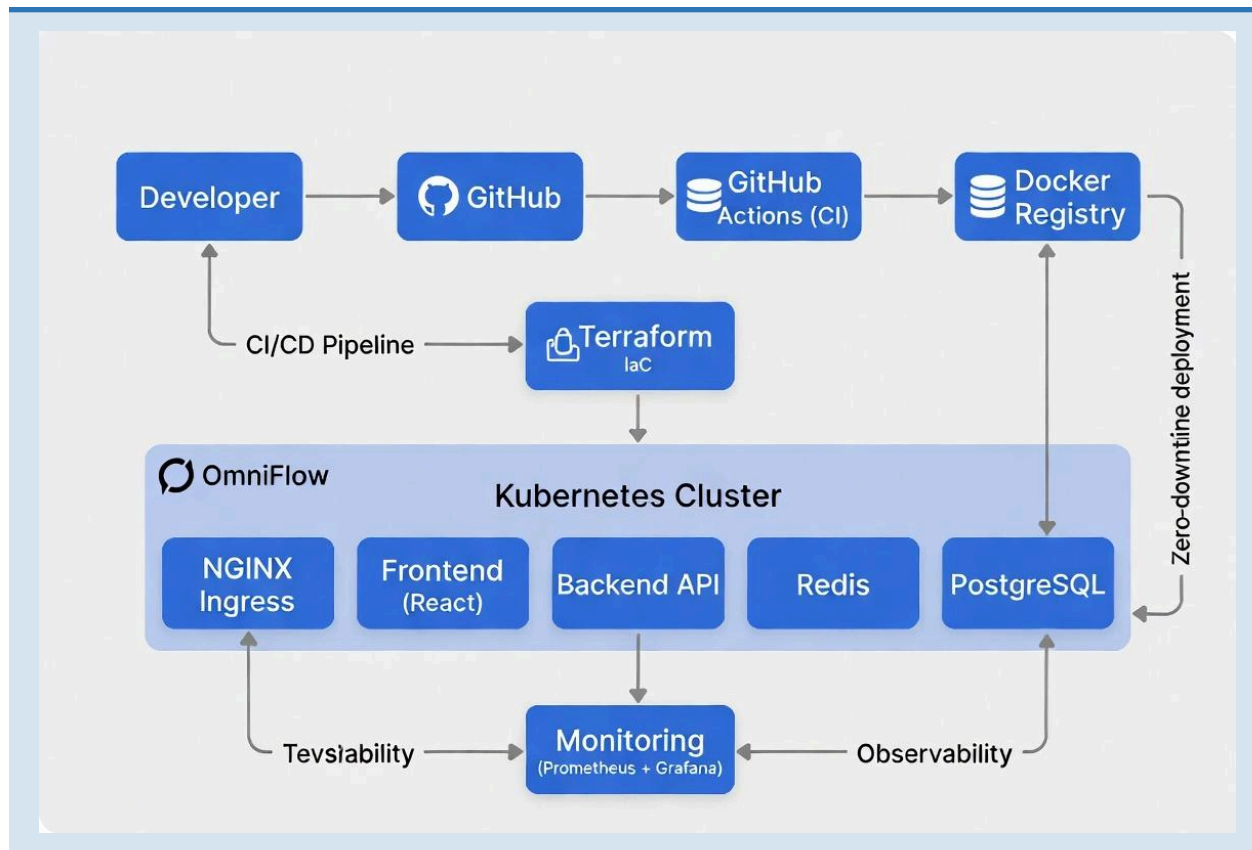
8.6 Disaster Recovery

The PostgreSQL database receives automated daily backups via the cloud provider. The entire environment can be destroyed and fully recreated from Terraform in under 15 minutes, meeting enterprise RTO requirements without manual runbooks.

9. Repository Structure & Deliverables

Submit the following directory structure in the project Git repository to demonstrate the full scope of the implementation.

Directory / File	Contents
.github/workflows/	CI Pipeline (GitHub Actions YAML files)
jenkins/	CD Pipeline (Jenkinsfile)
terraform/	IaC (main.tf, variables.tf, modules/)
helm/	Helm Charts (Chart.yaml, templates/, values.yaml)
src/	Simple Application Code (frontend/, backend/)
monitoring/	Helm values for Prometheus/Grafana/Loki & Dashboard JSONs
README.md	Project Documentation (Architecture diagrams, pipeline flow)



End of Document